VECTRONYX AI WORKBENCH PROJECT

Project Description

Vectronyx AI Workbench is a production-grade, modular AI platform that combines **data analysis, autonomous research, and contextual memory** into a unified intelligent system.

The platform allows users to interact with their data and queries using natural language, while internally orchestrating multiple AI modules such as a **data analyst engine, research agent, and memory-aware reasoning system**.

Unlike traditional AI tools, this system is designed with a **system-first approach**, incorporating **semantic memory retrieval, agent-based workflows, and dynamic query routing** to deliver context-aware, explainable insights.

It leverages open-source LLMs like Llama 3 via Ollama, vector similarity search using FAISS, and structured storage with Supabase (PostgreSQL), ensuring a fully cost-free and scalable architecture.

🎯 Project Goal

The goal of AI Workbench is to build a **real-world AI system** that demonstrates:

🧠 1. Intelligent System Design

- Modular architecture (not monolithic scripts)
- Query routing across multiple AI capabilities

📊 2. AI + Data Science Integration

- Natural language → data insights
- Automated EDA and visualization

🌐 3. Agent-Based Reasoning

- Autonomous research using multi-step workflows
- Information extraction and summarization

🧠 4. Persistent Memory System

- Long-term contextual awareness
- Personalized AI responses using semantic retrieval

⚙️ 5. Production Readiness

- Scalable backend design
- Containerization and CI/CD support

🧩 Core Utilities / Capabilities

📊 1. AI Data Analyst

What it does:

- Accepts CSV datasets
- Performs automated analysis
- Answers questions like:
    - “Why is revenue dropping?”
    - “Show trends over time”

Tech Used:

- Pandas
- Matplotlib
- Scikit-learn

🌐 2. Autonomous Research Agent

What it does:

- Takes a topic/query
- Searches, scrapes, and processes web data
- Generates structured insights/reports

Key Capabilities:

- Multi-step reasoning (ReAct pattern)
- Content summarization
- Insight generation

🧠 3. Contextual Memory System (USP)

What it does:

- Stores user interactions
- Retrieves relevant past context
- Improves response quality over time

How it works:

- Text → Embeddings → Stored in vector DB
- Query → Similarity search → Context injection

🔀 4. Intelligent Query Router

What it does:

- Understands user intent
- Routes request to:
  - Data module
  - Research agent
  - Memory system

🤖 5. AI Reasoning Engine

What it does:

- Combines:
  - User query
  - Retrieved memory
  - Module outputs
- Generates final response using LLM

🔐 6. User Management & Persistence

What it does:

- Authentication & session management
- User-specific data isolation

1. Vision & Problem Statement

Problem

Modern workflows are fragmented:

- Data analysis → separate tools
- Research → manual + time-consuming
- Context → lost between sessions

👉 No unified **AI-native system** that:

- Understands data
- Remembers users
- Performs autonomous reasoning

💡 **Solution**

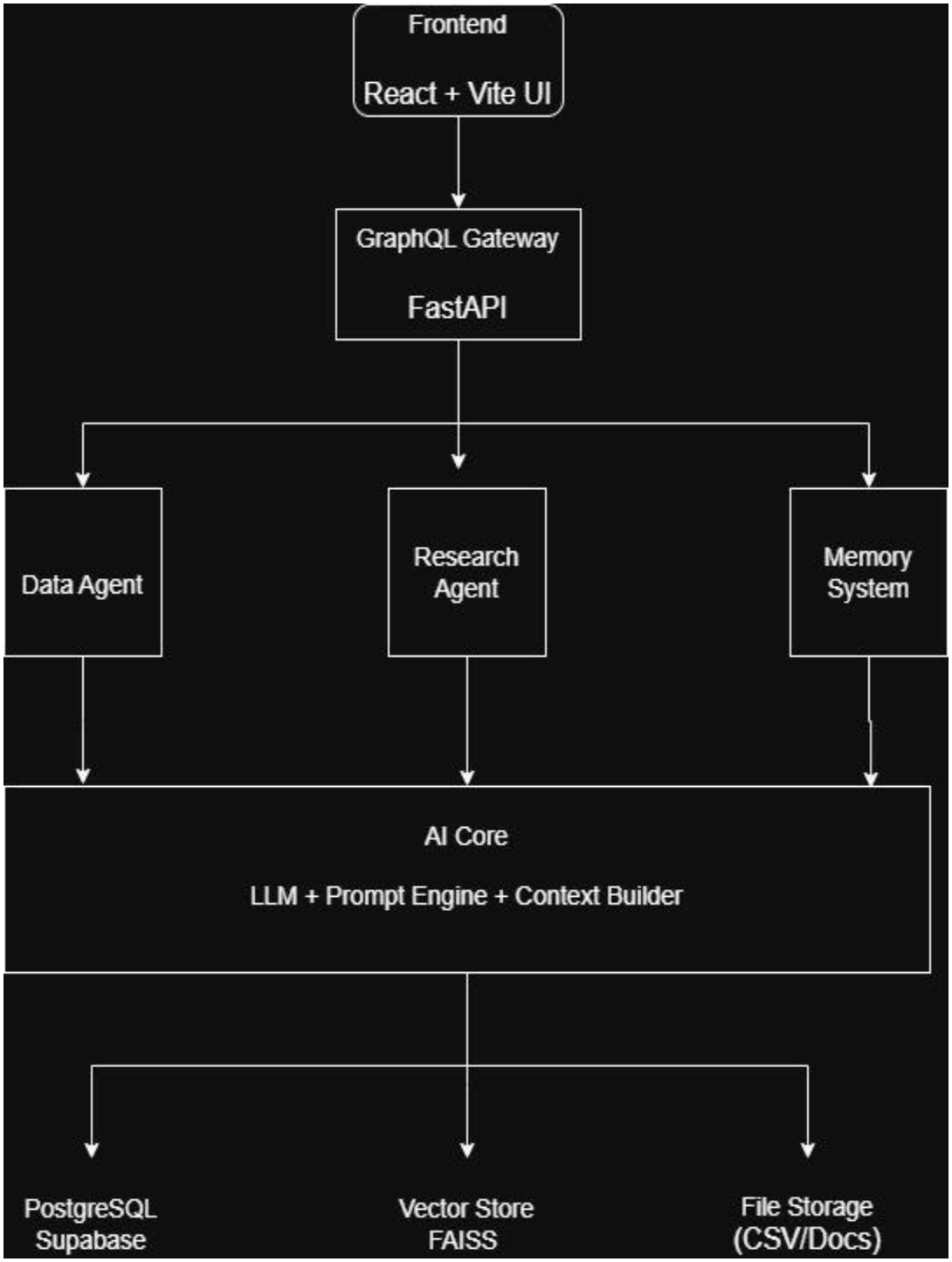
AI Workbench is a **modular intelligent system** that:

- Understands natural language
- Routes tasks intelligently
- Maintains long-term memory
- Executes multi-step reasoning

Core Philosophy

> “Not just AI features — but an AI system.”

2. High-Level Architecture (HLD)



3. Core Modules (L1 + L2 Breakdown)

1. 🧠 Query Router (Brain)

- Classifies intent:
  - Data
  - Research
  - Memory
- Routes request to module

👉 Use:

- Rule-based + lightweight classifier

2. 📊 **Data Analyst Module**

- CSV ingestion
- Auto EDA
- Natural language → Pandas queries
- Visualization

Algorithms:

- Descriptive statistics
- Trend detection
- Isolation Forest (anomaly detection)

3. 🌐 Research Agent (Agentic AI)

Pipeline:

User Query → Search → Scrape → Clean → Summarize → Report

Concepts:

- ReAct pattern
- Iterative reasoning loop

4. 🧠 Memory System (CORE USP)

Responsibilities:

- Store conversations
- Embed content
- Retrieve relevant memory

Pipeline:

Input → Embedding → Store

Query → Embedding → ANN Search → Top-K → Context

5. 🤖 AI Core

Responsibilities:

- Prompt construction
- Context injection
- Response generation

4. Low-Level Design (LLD - Critical)

Memory Retrieval Logic :

```
query_embedding = embed(query)

results = faiss.search(query_embedding, k=3)

context = [memory.text for memory in results]

final_prompt = f"""

Context:

{context}

User Query:

{query}

"""
```

🔀 Query Routing Logic :

```
if "csv" in query:

route = "data"

elif "why" in query:

route = "research"

else:

route = "memory"
```

5. Tech Stack (Finalized)

🧠 AI / LLM

- Llama 3
- Ollama

📊 Data Science

- Pandas
- Matplotlib
- Scikit-learn

🧠 Embeddings & Vector DB

- SentenceTransformers (MiniLM)
- FAISS

🗄️ Database

- Supabase (PostgreSQL)

🌐 Backend

- FastAPI
- GraphQL (Strawberry)

💻 Frontend

- React + Vite
- Tailwind CSS

⚙️ DevOps

- Docker
- GitHub Actions (CI/CD)

6. FINAL PROJECT STRUCTURE

```
Vectronyx/
│
├── backend/
│   ├── app/
│   │   ├── core/ # config, env, settings
│   │   │   ├──config.py
│   │   │   ├──constants.py
│   │   │   ├──dependencies.py
│   │   ├── api/ # GraphQL schema + resolvers
│   │   │   ├──schema.py #root GraphQL schema
│   │   │   ├──resolvers/
│   │   │   │   ├──memory_resolver.py
│   │   │   │   ├──data_resolver.py
│   │   │   │   ├──research_resolver.py
│   │   │   ├──types/ #GraphQL strawberry types
│   │   ├── router/ # query routing
│   │
│   │   ├── middleware/ # ✅ ADD THIS
│   │   │   ├── logging.py
│   │   │   ├── error_handler.py
│   │   │   ├── auth.py
│   │
│   │   ├── modules/
│   │   │   ├── data_analyst/
│   │   │   │   ├──__init__.py
│   │   │   │   ├──analyst.py
```

```
│   │   │   ├──prompts.py
│   │   │   ├──schemas.py
│   │   ├── research_agent/
│   │   │   ├──__init__.py
│   │   │   ├──agent.py
│   │   │   ├──tools.py
│   │   │   ├──prompts.py
│   │   │   ├──schemas.py
│   │   ├── memory/
│   │   │   ├──__init__.py
│   │   │   ├──embedder.py
│   │   │   ├──memory_store.py
│   │   │   ├──schemas.py
│   │
│   ├── services/
│   │   ├── llm/
│   │   │   ├──ollama_client.py  # Ollama API wrapper
│   │   │   ├──prompt_builder.py # context injection logic
│   │   ├── embeddings/
│   │   │   ├──sentence_transformer.py  # MiniLM wrapper
│   │   ├── vector_store/
│   │   │   ├──faiss_store.py    # FAISS index init, add, search
│   │
│   ├── db/
│   │   ├── models/
│   │   │   ├──user.py
│   │   │   ├──memory.py
│   │   │   ├──query_log.py
│   │   ├── repositories/ # ✅ clean DB access layer
│   │   │   ├──user_repo.py
```

```
│   │   │   ├──memory_repo.py
│   │   │   ├── migrations/
│   │   │   ├──database.py
│   │   │   ├──base.py
│   │
│   │   ├── auth/
│   │   ├── utils/
│   │
│   ├── tests/
│   ├── Dockerfile # ✅ backend container
│   ├── requirements.txt
│
├── frontend/
│   ├── src/
│   │   ├── app/ # routing setup
│   │   │   ├── router.tsx
│   │   ├── features/
│   │   │   ├── data-analyst/
│   │   │   ├── research/
│   │   │   ├── memory/
│   │   ├── components/
│   │   │   ├── ui/
│   │   │   ├── layout/
│   │   ├── graphql/ # queries + mutations
│   │   │   ├── queries/
│   │   │   ├── mutations/
│   │   ├── services/ # API client
│   │   │   ├── apollo-client.ts
│   │   ├── hooks/
│   │   ├── pages/
```

```
│   │   ├── store/
│   │   ├── types/
│   │   ├── constants/
│   │
│   ├── Dockerfile # ✅ ADD THIS
│   ├── vite.config.ts
│   ├── package.json
│
├── infra/
│   ├── docker-compose.yml # ✅ full stack
│   ├── docker-compose.dev.yml
│   ├── nginx/ # ✅ reverse proxy (optional but 🔥)
│   │   └── nginx.conf
│   ├── k8s/ (optional later)
│
├── scripts/
├── data/
├── vector_store/
│
├── .env # shared config
├── .github/workflows/
├── README.md
```

Final Architecture Flow :

Frontend (React)
↓
Apollo Client

↓
GraphQL (FastAPI)
↓
Middleware (auth, logging)
↓
Query Router
↓
Modules (Data / Research / Memory)
↓
LLM + FAISS + DB

7. GANTT CHART (20-DAY PLAN)

Day 1-2 : Project Setup + Repo + Env

Day 3-4 : Backend Boilerplate (FastAPI + GraphQL)

Day 5-6 : Memory System (FAISS + Embeddings)

Day 7-8 : Data Analyst Module

Day 9-10 : Research Agent

Day 11 : Query Router

Day 12 : LLM Integration

Day 13 : Frontend Setup

Day 14-15 : UI + API Integration

Day 16 : Auth (Supabase)

Day 17 : Testing

Day 18 : Dockerization

Day 19 : CI/CD

Day 20 : Final polish + README

8. Execution Strategy (VERY IMPORTANT)

🧠 Rule:

> Build vertical slices, not horizontal layers

❌ **Wrong**

- Build all backend first
- Then frontend

✅ **Correct**

- Build feature end-to-end:
  - Memory → UI → API → LLM

9. What Makes This Project ELITE

🔥 **1. Multi-domain integration**

- AI + Data + Agents + System Design

🔥 **2. Real-world architecture**

- Not a toy project

🔥 **3. Memory system (rare skill)**

- Most candidates don’t have this

🔥 **4. Agent-based reasoning**

- Current industry trend

Key Highlights

- Designed a **memory-augmented AI system** using semantic search
- Implemented **agentic AI workflows** with tool-based reasoning
- Built a **modular backend architecture** using FastAPI + GraphQL
- Integrated **vector databases (FAISS)** with relational DB (PostgreSQL)
- Developed a **natural language interface for data analytics**

One-Line Pitch

AI Workbench is a modular AI system that combines data analytics, autonomous research, and memory-aware reasoning into a single intelligent platform using LLMs and vector search.